SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Artificial Intelligence COURSE CODE: CSA17

QUESTIONS: 3 Duration: 1 Hour Total Marks: 30

COURSE OUTCOME COVERED

CO1: *Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively.* ***(BL3)***

Assessment Tool Weightage: 50%

Annexure A

Constraint-Based Problem Solving

Code of Conduct:

I S. Karthikeyan / 192419048 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Annexure A

CSA17 — ARTIFICIAL INTELLIGENCE

Assessment Tool 2 — Constraint-Based Problem Solving: Answers

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively

---

Q1. Doctor Shift Scheduling — Backtracking Search

Assign 3 doctors (D1, D2, D3) to 3 shifts (Morning, Afternoon, Night), one doctor per shift, subject to: D1 ≠ Night; D2 must be before D3 in the order Morning < Afternoon < Night; D3 ≠ Morning; all doctors assigned exactly one shift.

CSP Formulation

Variables: D1, D2, D3

Domain (each variable): {Morning, Afternoon, Night}

Constraints: D1 ≠ Night; D3 ≠ Morning; order(D2) < order(D3) where order(Morning)=1, order(Afternoon)=2, order(Night)=3; all-different(D1, D2, D3) — one doctor per shift.

Backtracking Search Trace (variable order D1 → D2 → D3; value order Morning → Afternoon → Night)

Step	Assignment Tried	Constraint Check	Result
1	D1 = Morning	D1 ≠ Night → satisfied	Accepted, proceed to D2
2	D2 = Morning	All-different: Morning already used by D1	FAIL — backtrack
3	D2 = Afternoon	All-different: Afternoon free; no direct constraint on D2 alone	Accepted, proceed to D3
4	D3 = Morning	All-different: Morning used by D1; also violates D3 ≠ Morning	FAIL — backtrack
5	D3 = Afternoon	All-different: Afternoon used by D2	FAIL — backtrack
6	D3 = Night	All-different: Night free. D3≠Morning: OK. order(D2)=2 < order(D3)=3: OK. D1≠Night: D1=Morning, OK.	ALL CONSTRAINTS SATISFIED — SOLUTION

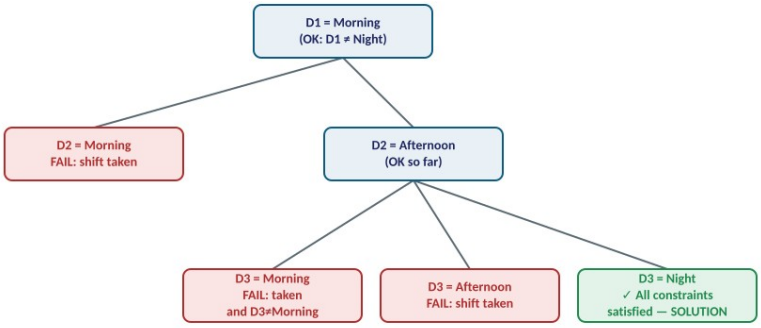

Figure 1 — Backtracking search tree: red nodes are pruned by a failed constraint check, green is the accepted solution.

Final Valid Schedule

Doctor	Assigned Shift
D1	Morning
D2	Afternoon
D3	Night

Note: *A second valid schedule also exists — D1 = Afternoon, D2 = Morning, D3 = Night — which likewise satisfies every constraint (D1≠Night ✓, D3≠Morning ✓, order(D2)=1<order(D3)=3 ✓). Standard backtracking with left-to-right value ordering (Morning→Afternoon→Night) finds D1=Morning, D2=Afternoon, D3=Night first, which is reported above as the answer, since the question asks for "a valid assignment."*

Q2. Robot in a 5×5 Grid — BFS Search

Note: *The obstacle layout and exact S/G positions were not included in the uploaded file (no diagram accompanied this question). A representative 5×5 grid is assumed below — S at the top-left corner (0,0), G at the bottom-right corner (4,4), and four obstacles positioned to require real detours — so the full BFS method is demonstrated step-by-step. Re-run the same procedure on your actual grid if it differs.*

Assumed Grid (rows/cols 0-indexed)

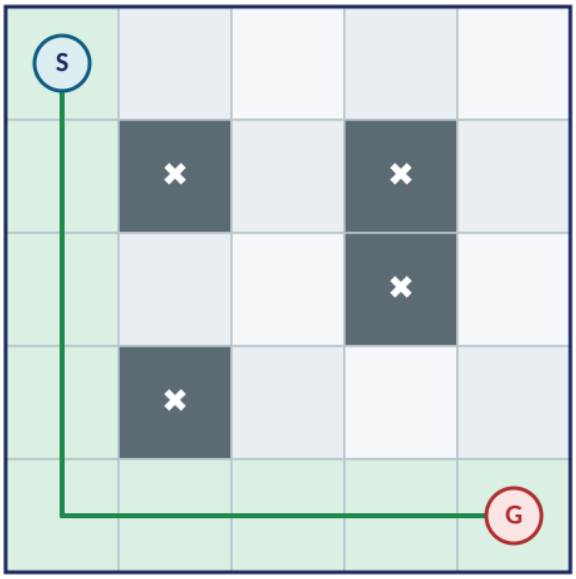

Figure 2 — 5×5 grid. S = start (0,0), G = goal (4,4), dark cells = obstacles at (1,1), (1,3), (2,3), (3,1). Green = optimal path found by BFS.

Method

BFS explores the grid level-by-level using a FIFO frontier queue, expanding all nodes at distance k before any node at distance k+1. Since every move has an identical cost of 1, BFS is guaranteed to find the shortest (least-cost) path here — it coincides exactly with what Uniform Cost Search would return for this uniform-cost grid. (Note: classic BFS itself does not consult a heuristic — the Manhattan-distance heuristic is normally used to guide informed searches such as A*/Greedy Best-First. It is reported alongside each node below purely for reference, showing how much closer to G each step gets.)

Step-by-Step Node Expansion and Queue Updates

#	Node Expanded (r,c)	g(n) cost so far	h(n) Manhattan to G	New Nodes Added to Frontier
1	(0,0) — S	0	8	(1,0), (0,1)
2	(1,0)	1	7	(2,0) [(1,1) is an obstacle]
3	(0,1)	1	7	(0,2) [(1,1) is an obstacle]
4	(2,0)	2	6	(3,0), (2,1)
5	(0,2)	2	6	(1,2), (0,3)
6	(3,0)	3	5	(4,0) [(3,1) is an obstacle]
7	(2,1)	3	5	(2,2) [(1,1),(3,1) are obstacles]
8	(1,2)	3	5	— (all neighbours visited/obstacle)
9	(0,3)	3	5	(0,4) [(1,3) is an obstacle]
10	(4,0)	4	4	(4,1)
11	(2,2)	4	4	(3,2) [(2,3) is an obstacle]
12	(0,4)	4	4	(1,4)
13	(4,1)	5	3	(4,2) [(3,1) is an obstacle]
14	(3,2)	5	3	(3,3) [(4,2) already queued]
15	(1,4)	5	3	(2,4) [(1,3) is an obstacle]
16	(4,2)	6	2	(4,3)
17	(3,3)	6	2	(3,4) [(2,3) is obstacle; (4,3) already queued]
18	(2,4)	6	2	— (all neighbours visited/queued)

19	(4,3)	7	1	(4,4) = G reached — GOAL

Result

Optimal path: (0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

Optimal cost: 8 moves (matches g(n) = 8 at the goal, and BFS's search depth of 8 — confirming optimality since all edge costs are equal).

Q3. Autonomous Rescue Robot — Partially Observable, Online Search

A rescue robot must reach a survivor at G from S in a grid of rooms, with obstacles, limited (adjacent-cell only) sensing, move cost 1 (2 in risky zones), while minimising total cost, navigating safely, and updating its knowledge dynamically as it goes.

(a) How Each Constraint Affects Decision-Making

- 4-directional movement — caps the branching factor at 4 per cell, keeping the local search the robot must run at every step computationally cheap enough for real-time, onboard decision-making.
- Blocked cells (obstacles) — remove certain transitions from the state graph; since they are not all known in advance, the robot must treat them as discovered constraints, only ruling them out once actually sensed rather than assuming a complete map upfront.
- Limited sensing (adjacent cells only) — makes the environment partially observable from the robot's perspective. It cannot compute one global optimal plan at the start; it can only decide with confidence about cells it has already sensed, forcing an interleaved sense-plan-act cycle.
- Variable move cost (1 normal, +2 in risky zones) — turns this into a weighted shortest-path problem rather than a simple unweighted one, so the robot must prefer a longer-but-cheaper/safer route over a shorter-but-riskier one whenever the total cost favours it.
- Minimise total cost while ensuring safe navigation — a dual objective; folding the safety penalty directly into the move cost (the +2 for risky zones) lets a single cost-minimising search still produce safe behaviour, without needing two separate objectives to balance.
- Dynamic knowledge updates (online search) — rules out any purely offline algorithm that plans once and executes blindly; the robot must be able to revise its plan whenever newly sensed information (e.g., a previously unknown obstacle) contradicts its current assumptions.

(b) Solution Approach: Online (Adaptive) Uniform Cost Search

Because the map is only partially known, classic offline UCS (which assumes the whole weighted graph is known in advance) cannot be applied directly. Instead, the robot runs an online search agent that interleaves sensing, local re-planning (using UCS over its currently known map), and acting, in a repeating loop:

- Step 1 — Sense: from the current cell, observe all adjacent cells (safe / obstacle / risky-zone), and update the robot's internal partial map with this new information.
- Step 2 — Local Plan: run Uniform Cost Search over the currently known portion of the map from the current cell, treating any still-unexplored cell optimistically as normal-cost and traversable (a standard assumption in online search agents), to choose the next move that minimises known cumulative cost toward G.
- Step 3 — Act: execute that single next move (or a short lookahead sequence) rather than the whole plan, since the plan may need to change once more of the map is revealed.
- Step 4 — Update & Repeat: incorporate whatever the move revealed (e.g., a cell that turned out to be a risky zone) into the internal map, and repeat Steps 1–3 from the new position until G is reached.
- Because UCS always expands the lowest cumulative-cost frontier node and every move cost is non-negative, each local re-plan remains cost-optimal given current knowledge — so as the robot explores, its running path converges to the true minimum-cost route to the survivor.

(c) Demonstrating Core AI Concepts

Intelligent agent: the robot fits the standard agent loop — Percepts (states of adjacent cells), Actions (move up/down/left/right), Goal (reach the survivor safely and cheaply), Environment (the disaster-affected building). It

continually senses its surroundings and acts to change its state, rather than following one fixed, pre-computed sequence.

Rationality: the robot is rational because, given everything it has perceived so far and its built-in cost model, it always chooses the action expected to minimise total cost while avoiding known hazards — i.e., it maximises its expected performance measure given its percept history, without requiring impossible, complete knowledge of the whole building in advance (rationality is about doing the best one can with available information, not omniscience).

Environment type: partially observable (only adjacent cells are sensed), sequential (each move affects future options and knowledge), dynamic in the agent's own knowledge as exploration proceeds (even though the physical building itself does not move), effectively stochastic/uncertain from the robot's subjective view (it cannot be sure a cell is safe until it senses it), and discrete (a finite grid of rooms and a finite action set).

(d) Justification for the Chosen Approach

- Plain offline UCS or A* is not viable, since both assume the full weighted map is known before search begins — this directly conflicts with the limited-sensing and "update knowledge dynamically" constraints.
- Plain BFS is not appropriate, since it assumes every move has equal cost, but this problem explicitly has two different move costs (1 normal vs. 2 risky) — a cost-blind algorithm could select a technically shorter path that is actually more expensive or more dangerous.
- An online, cost-aware search directly satisfies every stated constraint at once: it tolerates unknown obstacles (dynamic knowledge update), respects variable move costs (via UCS's cumulative-cost-ordered expansion), and needs only local knowledge to decide each next step (matching the robot's limited sensing radius).
- For these reasons, an Online (adaptive) Uniform Cost Search agent — interleaving sensing, local re-planning, and acting — is the most suitable and best-justified approach for this specific combination of partial observability, variable cost, and dynamic updates.